



Auto ▾



Home



Explore



Notifications



Chat



Grok



Premium



Bookmarks



Creator Studio



Articles



Profile



More

TheRocketMan 
@yeobuzzfx

...



Ask anything

analyze the python strategy

"""

Trading strategy: run_cycle on each new bar, custom_math hook, position sizing, risk flatten.

Decision windows: run signals 5 minutes before and 5 minutes at the 20- and 50-minute marks each hour (minutes 15, 20, 25, 45, 50, 55). Works for stocks, cryp and any Alpaca asset.

"""

```
from __future__ import annotations
```

```
import json
```

```
import logging
```

```
import threading
```

```
from collections import defaultdict
```

```
from datetime import datetime, timezone
```

```
from pathlib import Path
```

```
from queue import Queue
```

```
from typing import Any
```

```
from broker import AlpacaBroker, BarData, asset_type
```

```
from config import Config
```

```
logger = logging.getLogger(__name__)
```

```
TRADES_DIR = Path(__file__).resolve().parent / "trades"
```

```
# Optional custom math: returns "buy" | "sell" | "hold"
```

```
_custom_math_loaded = False
```

```
_your_signal_function = None
```

```
try:
```

```
    from custom_math import get_signal_context
```

```
except ImportError:
```

```
    get_signal_context = None # type: ignore[misc, assignment]
```

```
def _load_custom_math() -> bool:
```

```
    global _custom_math_loaded, _your_signal_function
```

```
    if _custom_math_loaded:
```

```
        return _your_signal_function is not None
```

```
    _custom_math_loaded = True
```

```
    try:
```

```
        from custom_math import your_signal_function
```

```
        _your_signal_function = your_signal_function
```

```
        logger.info("Custom math loaded: your_signal_function")
```

```
        return True
```

```
    except ImportError as e:
```

```
        logger.error("Custom math not used (import failed): %s", e)
```

```
        return False
```

```

class TradingStrategy:
    """
    Runs on each new bar: custom math signal, position sizing, risk
    check (flatten if unrealized loss >= max %).
    Pushes log/UI messages via ui_queue (dicts with type/text/color
    type/data).
    """

    def __init__(
        self,
        broker: AlpacaBroker,
        watched_symbols: list[str],

        config: Config,
        ui_queue: Queue[dict[str, Any]] | None = None,
    ) -> None:
        self.broker = broker
        self.watched_symbols = watched_symbols
        self.config = config
        self.ui_queue = ui_queue or Queue()
        self._lock = threading.Lock()
        self._running = True
        self._bar_buffer: dict[str, BarData] = {}
        self._bar_history: dict[str, list[BarData]] = {}
        self._position_entry_time: dict[str, datetime] = {} # normalized
        symbol -> UTC entry time
        self._asset_class_buy_rotation: list[str] = ["stock", "crypto"] # orc
        for preferring which class to buy
        self._last_buy_class_index: int = 0 # next class to prefer when
        multiple buys
        # Eager-load signal so first bar can trade immediately
        _load_custom_math()
        if _your_signal_function is not None:
            self._log_ui("Strategy ready. Trading on every bar.", "info")
        else:
            self._log_ui("Strategy ready but custom_math not loaded — n
            trades until signal is available.", "blue")

    def _log_ui(self, text: str, color: str = "info") -> None:
        if self.ui_queue is not None:
            self.ui_queue.put({"type": "log", "text": text, "color": color})

    def start(self) -> None:
        """Resume strategy after stop(); allows run_cycle to process bar
        again."""
        self._running = True

    def _can_trade(self) -> bool:
        """Check position count, account, and daily loss limit."""
        try:
            if self.daily_loss_limit_exceeded():

```

```

    if self._daily_loss_limit_reached():
        self._log_ui("No new trades: daily loss limit reached.", "blue")
        return False
    positions = self.broker.get_positions()
    if len(positions) >= self.config.MAX_POSITIONS:
        self._log_ui(f"No new trades: max positions ({self.config.MAX_POSITIONS}) reached.", "blue")
        return False
    return True
except Exception:
    return False

def _compute_qty(self, symbol: str, side: str) -> float:
    """Position size: fixed qty or % of equity; capped by buying_power
    MARGIN_SAFETY_FRACTION."""
    if self.config.FIXED_QTY is not None:
        base_qty = float(self.config.FIXED_QTY)
    else:
        try:
            account = self.broker.get_account()
            if account is None:
                return 1.0
            equity = float(getattr(account, "equity", 0) or 0)
            if equity <= 0:
                return 1.0

            pct = self.config.POSITION_SIZE_PCT / 100.0
            notional = equity * pct
            bar = self._bar_buffer.get(symbol)
            price = float(bar.close) if bar else 1.0
            if price <= 0:
                return 1.0
            qty = notional / price
            base_qty = max(1, round(qty, 0)) if qty >= 1 else 1.0
        except Exception as e:
            logger.debug("_compute_qty: %s", e)
            return 1.0

    # Cap by margin: order notional must not exceed buying_power
    safety_fraction
    try:
        buying_power = self.broker.get_buying_power()
        if buying_power is not None and buying_power > 0:
            safety = getattr(self.config, "MARGIN_SAFETY_FRACTION", 1)
            max_notional = buying_power * safety
            bar = self._bar_buffer.get(symbol)
            price = float(bar.close) if bar else 1.0
            if price > 0:
                max_qty = max_notional / price
                if base_qty * price > max_notional:
                    base_qty = max(1, int(max_qty)) if max_qty >= 1 else (ma
    if asset_type(symbol) == "crypto" else 1.0)
            logger.debug("_compute_qty: reduced qty for margin

```

```

        logger.debug("_compute_qty: reduced qty for margin
(symbol=%s)", symbol)
    except Exception as e:
        logger.debug("_compute_qty margin cap: %s", e)
    return base_qty

```

```

def _daily_loss_limit_breached(self) -> bool:
    """True if day PnL is worse than MAX_DAILY_LOSS_PCT of previc
equity.
    Uses Alpaca last_equity (previous trading day); if missing we us
equity so day_pnl_pct is 0 and limit never triggers.
    """

```

```

    try:
        account = self.broker.get_account()
        if account is None:
            return False
        equity = float(getattr(account, "equity", 0) or 0)
        last_equity = float(getattr(account, "last_equity", 0) or 0) or ec
        if last_equity <= 0:
            return False
        day_pnl_pct = (equity - last_equity) / last_equity * 100.0
        max_daily = getattr(self.config, "MAX_DAILY_LOSS_PCT", 5.0)
        return day_pnl_pct <= -max_daily
    except Exception:
        return False

```

```

def _record_position_entry_now(self, sym: str) -> None:
    """Set position entry time to now for symbol (first sight or new
buy)."""
    key = self._normalize_symbol_key(sym)
    self._position_entry_time[key] = datetime.now(timezone.utc)

```

```

def _clear_position_entry(self, sym: str) -> None:
    """Remove entry time when position is closed."""
    key = self._normalize_symbol_key(sym)
    self._position_entry_time.pop(key, None)

```

```
@staticmethod
```

```

def _bar_to_dict(bar_data: BarData | None) -> dict[str, Any] | None:
    """Serialize bar for trade record; None if bar_data is None."""
    if bar_data is None:
        return None
    return {
        "open": float(getattr(bar_data, "open", 0) or 0),
        "high": float(getattr(bar_data, "high", 0) or 0),
        "low": float(getattr(bar_data, "low", 0) or 0),
        "close": float(getattr(bar_data, "close", 0) or 0),
        "volume": float(getattr(bar_data, "volume", 0) or 0),
        "timestamp": str(getattr(bar_data, "timestamp", "") or ""),
    }

```

```

def _record_trade(
    self,
    symbol: str,
    side: str,
    qty: float,
    trigger: str,
    bar_data: BarData | None = None,
    unrealized_pl_pct: float | None = None,
    signal_context: dict[str, Any] | None = None,
) -> None:
    """Append one trade record to trades/trades_YYYY-MM-DD.json
    future learning."""
    try:
        ts_utc = datetime.now(timezone.utc).isoformat()
        bar = self._bar_to_dict(bar_data)
        record = {
            "ts_utc": ts_utc,

            "symbol": symbol,
            "side": side,
            "qty": qty,
            "trigger": trigger,
            "bar": bar,
            "unrealized_pl_pct": unrealized_pl_pct,
            "signal_context": signal_context,
        }
        TRADES_DIR.mkdir(exist_ok=True)
        path = TRADES_DIR /
f"trades_{datetime.now(timezone.utc).strftime('%Y-%m-%d')}.json"
        with self._lock:
            with path.open("a", encoding="utf-8") as f:
                f.write(json.dumps(record, default=str) + "\n")
    except Exception as e:
        logger.debug("_record_trade failed: %s", e)

def _risk_check(self) -> None:
    """Flatten by stop-loss / take-profit; flatten all if daily loss limit
    breached. No time-based max-hold."""
    try:
        positions = self.broker.get_positions()
        if self._daily_loss_limit_breached():
            for pos in positions:
                sym = getattr(pos, "symbol", None) or getattr(pos, "symbol", None)
            qty = float(getattr(pos, "qty", 0) or 0)
            if not sym or qty == 0:
                continue
            side = self._position_side(pos)
            close_side = "sell" if side == "long" else "buy"

```

```

        self.broker.submit_order(sym, abs(qty), close_side)
        self._record_trade(sym, close_side, abs(qty), "daily_loss",
bar_data=None, signal_context=None)
        self._log_ui(f"Daily loss limit: flattened {side} {abs(qty)} {s}
"red")
        self._clear_position_entry(sym)
        return
        max_loss_pct = getattr(self.config, "MAX_UNREALIZED_LOSS_
5.0)
        take_profit_pct = getattr(self.config, "TAKE_PROFIT_PCT", 0.0)
        positions = self.broker.get_positions()
        for pos in positions:
            sym = getattr(pos, "symbol", None) or getattr(pos, "symbol_
""")
            if not sym:
                continue
            unrealized_pct = float(getattr(pos, "unrealized_plpc", 0) or C
100.0
            qty = float(getattr(pos, "qty", 0) or 0)
            if qty == 0:
                continue
            side = self._position_side(pos)
            close_side = "sell" if side == "long" else "buy"
            if unrealized_pct <= -max_loss_pct:
                self.broker.submit_order(sym, abs(qty), close_side)
                self._record_trade(sym, close_side, abs(qty), "risk_flatten",
bar_data=None, unrealized_pl_pct=unrealized_pct,
signal_context=None)
                self._clear_position_entry(sym)
                self._log_ui(f"Risk flatten: closed {side} {abs(qty)} {sym} (l
%={unrealized_pct:.2f})", "red")

            elif take_profit_pct > 0 and unrealized_pct >= take_profit_pc
                self.broker.submit_order(sym, abs(qty), close_side)
                self._record_trade(sym, close_side, abs(qty), "take_profit",
bar_data=None, unrealized_pl_pct=unrealized_pct,
signal_context=None)
                self._clear_position_entry(sym)
                self._log_ui(f"Take profit: closed {side} {abs(qty)} {sym} (g
%={unrealized_pct:.2f})", "green")
        except Exception as e:
            logger.exception("_risk_check: %s", e)

    @staticmethod
    def _symbol_eq(a: str, b: str) -> bool:
        """True if symbols refer to same asset (e.g. BTC/USD and
BTCUSD)."""
        if a == b:
            return True
        return (a or "").replace("/", "").upper() == (b or "").replace("/",
""").upper()

```

```

@staticmethod
def _position_side(pos: Any) -> str:
    """Return 'long' or 'short' from Alpaca position (side attribute or
    sign)."""
    side = (getattr(pos, "side", None) or "").strip().lower()
    if side == "short":
        return "short"
    qty = float(getattr(pos, "qty", 0) or 0)
    if qty < 0:
        return "short"
    return "long"

```

```

@staticmethod
def _normalize_symbol_key(symbol: str) -> str:
    """Canonical key for position entry time dict."""
    return (symbol or "").replace("/", "").upper()

```

```

def _minute_of_hour(self, bar_data: BarData) -> int | None:
    """Parse bar timestamp; return minute (0-59) or None if
    unparseable."""
    ts = getattr(bar_data, "timestamp", "") or ""
    if not ts:
        return None
    try:
        if hasattr(bar_data.timestamp, "minute"):
            return bar_data.timestamp.minute
        s = str(ts).replace("Z", "+00:00")
        dt = datetime.fromisoformat(s)
        return dt.minute
    except Exception:
        return None

```

```

def run_cycle(self, bar_data_dict: dict[str, BarData]) -> None:
    """
    Called on every new bar (or batch). Updates bar buffer and hist
    runs risk check.
    Signal runs on every bar (no minute filter). Exits are signal- and
    based only.
    """
    if not bar_data_dict:
        return
    max_history = getattr(self.config, "BAR_HISTORY_MAX", 120)

```

```

    with self._lock:
        self._bar_buffer.update(bar_data_dict)
        for symbol, bar_data in bar_data_dict.items():
            hist = self._bar_history.setdefault(symbol, [])
            hist.append(bar_data)
            if len(hist) > max_history:

```

```

        if len(hist) > max_history:
            self._bar_history[symbol] = hist[-max_history:]

    def _risk_check():
    if not self._running:
        return
    can_trade = self._can_trade()
    focus_results: list[tuple[str, str]] = [] # (symbol, signal) for visibility
    log
    buy_symbols: list[str] = [] # symbols with buy signal and no long
    (open long)
    sell_symbols: list[str] = [] # symbols with sell signal and no short
    (open short)

    for symbol, bar_data in bar_data_dict.items():
        with self._lock:
            history = list(self._bar_history.get(symbol, []))

            signal = "hold"
            if _your_signal_function is not None:
                try:
                    signal = _your_signal_function(
                        symbol, bar_data, self.broker, bar_history=history,
                        config=self.config
                    )

                    if signal not in ("buy", "sell", "hold"):
                        signal = "hold"
                except Exception as e:
                    logger.exception("Custom math signal error for %s: %s",
                        symbol, e)
                    signal = "hold"
            else:
                _load_custom_math()
                if _your_signal_function is None:
                    continue

            focus_results.append((symbol, signal))
            positions = self.broker.get_positions()

            if signal == "buy":
                has_position = False
                for pos in positions:
                    psym = getattr(pos, "symbol", None) or getattr(pos,
                        "symbol_id", "")
                    if not self._symbol_eq(psym, symbol):
                        continue
                    has_position = True
                    qty = float(getattr(pos, "qty", 0) or 0)
                    if qty == 0:
                        continue
                    side = self._position_side(pos)

```



```

        order = self._position_order(pos)
        if side == "short":
            try:
                self.broker.submit_order(symbol, abs(qty), "buy")
                ctx = get_signal_context(symbol, bar_data, self.broker.history, self.config) if get_signal_context else None

                self._record_trade(symbol, "buy", abs(qty), "cover_short", bar_data=bar_data, signal_context=ctx)
                self._clear_position_entry(symbol)
                self._log_ui(f"Cover short {abs(qty)} {symbol}", "green")
            except Exception as e:
                self._log_ui(f"Cover failed {symbol}: {e}", "red")
            break
        if not has_position:
            buy_symbols.append(symbol)

    if signal == "sell":
        has_position = False
        for pos in positions:
            psym = getattr(pos, "symbol", None) or getattr(pos, "symbol_id", "")
            if not self._symbol_eq(psym, symbol):
                continue
            has_position = True
            qty = float(getattr(pos, "qty", 0) or 0)
            if qty == 0:
                continue
            side = self._position_side(pos)
            if side == "long":
                try:
                    self.broker.submit_order(symbol, abs(qty), "sell")
                    ctx = get_signal_context(symbol, bar_data, self.broker.history, self.config) if get_signal_context else None
                    self._record_trade(symbol, "sell", abs(qty), "sell_long", bar_data=bar_data, signal_context=ctx)
                    self._clear_position_entry(symbol)
                    self._log_ui(f"Sell long {abs(qty)} {symbol}", "red")

                except Exception as e:
                    self._log_ui(f"Sell failed {symbol}: {e}", "red")
                break
            if not has_position:
                sell_symbols.append(symbol)

    # At most one new long per cycle; prefer symbol from under-trading asset class (rotation)
    if can_trade and buy_symbols:
        positions = self.broker.get_positions()
        position_count_by_class: dict[str, int] = defaultdict(int)
        for pos in positions:
            psym = getattr(pos, "symbol", None) or getattr(pos, "symbol_id", "")

```

```

"""
    if psym:
        position_count_by_class[asset_type(psym)] += 1
    classes = self._asset_class_buy_rotation
    idx = self._last_buy_class_index % len(classes) if classes else C
    preferred_class = classes[idx]
    chosen = None
    for sym in buy_symbols:
        if asset_type(sym) == preferred_class:
            chosen = sym
            break
    if chosen is None:
        chosen = buy_symbols[0]
    else:
        self._last_buy_class_index = (idx + 1) % len(classes)
    qty = self._compute_qty(chosen, "buy")
    try:
        self.broker.submit_order(chosen, qty, "buy")

        with self._lock:
            hist_chosen = list(self._bar_history.get(chosen, []))
            bar_chosen = self._bar_buffer.get(chosen)
            ctx = get_signal_context(chosen, bar_chosen, self.broker,
hist_chosen, self.config) if get_signal_context and bar_chosen else
None
            self._record_trade(chosen, "buy", qty, "entry_long",
bar_data=bar_chosen, signal_context=ctx)
            self._record_position_entry_now(chosen)
            self._log_ui(f"Buy (long) {qty} {chosen}", "green")
    except Exception as e:
        self._log_ui(f"Buy failed {chosen}: {e}", "red")

# At most one new short per cycle; same asset-class rotation as
longs
if can_trade and sell_symbols:
    positions = self.broker.get_positions()
    position_count_by_class: dict[str, int] = defaultdict(int)
    for pos in positions:
        psym = getattr(pos, "symbol", None) or getattr(pos, "symbol", None)
"""
    if psym:
        position_count_by_class[asset_type(psym)] += 1
    classes = self._asset_class_buy_rotation
    idx = self._last_buy_class_index % len(classes) if classes else C
    preferred_class = classes[idx]
    chosen = None
    for sym in sell_symbols:
        if asset_type(sym) == preferred_class:
            chosen = sym
            break

```

```

if chosen is None:
    chosen = sell_symbols[0]
else:
    self._last_buy_class_index = (idx + 1) % len(classes)
qty = self._compute_qty(chosen, "sell")
try:
    self.broker.submit_order(chosen, qty, "sell")
    with self._lock:
        hist_chosen = list(self._bar_history.get(chosen, []))
        bar_chosen = self._bar_buffer.get(chosen)
        ctx = get_signal_context(chosen, bar_chosen, self.broker,
hist_chosen, self.config) if get_signal_context and bar_chosen else
None
        self._record_trade(chosen, "sell", qty, "entry_short",
bar_data=bar_chosen, signal_context=ctx)
        self._record_position_entry_now(chosen)
        self._log_ui(f"Sell (short) {qty} {chosen}", "red")
except Exception as e:
    self._log_ui(f"Short failed {chosen}: {e}", "red")

if focus_results:
    parts = [f"{s} {sig}" for s, sig in focus_results]
    trades = sum(1 for _, sig in focus_results if sig in ("buy", "sell"))
    msg = f"Focus: {' '.join(parts)}"
    if trades:
        msg += f" ({trades} trade(s))"
    self._log_ui(msg, "info")

def stop(self) -> None:
    self._running = False
    with self._lock:
        self._bar_buffer.clear()
        self._bar_history.clear()
        self._position_entry_time.clear()

```

Comprehensive Analysis of the TradingStrategy

This is a **lightweight, event-driven, bar-based trading bot** designed for Alpaca (stocks + crypto + other supported asset). It is deliberately simple, modular, and production-oriented with strong risk controls and detailed trade journaling for later ML/offline analysis.

1. High-Level Design & Philosophy

- **Core loop:** `run_cycle(bar_data_dict)` is called on **every new bar** (1-min, 5-min, whatever your feed pushes).
- **Signal source:** 100% delegated to an external `custom_math.your_signal_function(...)` → `"buy" | "sell" | "hold"`.
- **Position limit:** Max one new **long** + one new **short** per cycle (very conservative).
- **Risk-first:** Risk checks run **before** any new entries.
- **Diversification:** Simple round-robin preference between asset classes (`stock` → `crypto`).

- **Data-driven:** Every trade is logged as JSONL with full bar + signal context → perfect for backtesting / feature engineering later.

2. What Actually Runs vs. the Docstring

The top comment says:

"Decision windows: run signals 5 minutes before and 5 minutes after the 20- and 50-minute moving averages each hour (minutes 15, 20, 25, 45, 50, 55)"

This is NOT implemented in the code.

- `_minute_of_hour()` exists but is **never called**.
- Signals run on **every single bar**.
- This is probably the biggest mismatch between intention and implementation.

3. Detailed Flow (run_cycle)

```
python
for every symbol in this bar batch:
    signal = your_signal_function(...) # custom math hook

    if signal == "buy":
        if already short → cover it immediately
        elif no position → add to buy_candidates

    if signal == "sell":
        if already long → sell it immediately
        elif no position → add to sell_candidates

# After processing all symbols:
if can_trade and buy_candidates:
    pick ONE (with asset-class rotation preference)
    submit buy

if can_trade and sell_candidates:
    pick ONE (same rotation logic)
    submit short
```

Key behaviors:

- Opposite signal on existing position = **instant flatten** (cover short or sell long).
- Same-direction signal on existing position = **ignored** (no pyramiding).
- At most **one new long + one new short per bar** → excellent exposure control.

4. Risk Management (excellent)

Feature	Trigger	Action	Logged as
Daily loss limit	<code>day_pnl_pct <= -MAX_DAILY_LOSS_PCT</code>	Flatten ALL positions	<code>daily_loss</code>
Per-position stop-loss	<code>unrealized_plpc < -MAX_UNREALIZED_LOSS_PCT</code>	Flatten that position	<code>risk_flatten</code>

Take-profit

unrealized_plpc ≥ TAKE_PROFIT_PCT

Flatten that position

take_profi

- Runs **every cycle** before new entries.
- Uses Alpaca’s native `unrealized_plpc` (very reliable).
- No time-based exit (entry times are recorded but never used).

5. Position Sizing (`_compute_qty`)

Two modes:

- `FIXED_QTY` in config → simple fixed shares/contracts.
- Percentage of equity (`POSITION_SIZE_PCT`) → dynamic, recalculated every time.

Then **hard-capped** by:

```
python
max_notional = buying_power * MARGIN_SAFETY_FRACTION # default 0.95
```

Crypto gets fractional quantities; stocks are rounded to whole shares (sensible).

6. Asset-Class Rotation (clever but subtle)

```
python
_asset_class_buy_rotation = ["stock", "crypto"]
_last_buy_class_index      # cycles between 0 and 1
```

- When multiple buy (or sell) signals appear in the same cycle, it **prefers** the next class in the rotation.
- Only advances the index when it actually picks the preferred class.
- Same logic used for longs **and** shorts → tends to keep balance between stocks and crypto over time.

7. Trade Recording (gold for iteration)

Every fill is written to:

```
trades/trades_YYYY-MM-DD.jsonl
```

Fields include:

- `ts_utc`, `symbol`, `side`, `qty`
- Full bar OHLCV
- `unrealized_pl_pct` at close
- `signal_context` from `get_signal_context()` (if you implement it)

This is perfect for building your own reinforcement-learning dataset or walk-forward optimization.

8. Strengths

- Extremely clean separation of concerns (signal vs execution vs risk).
- Very safe defaults (max positions, margin safety, daily loss kill-switch).
- Handles long/short covering automatically.

manages long/short covering automatically.

- Thread-safe, production-ready logging + UI queue.
- Easy to swap signal logic (just edit `custom_math.py`).

9. Weaknesses / Bugs / Missing Pieces

Issue	Severity	Comment
Decision windows not implemented	High	Most important missing feature
<code>watched_symbols</code> stored but never used	Medium	Dead code
<code>_position_entry_time</code> recorded but never read	Low	Probably planned for time-based exit
No per-symbol max hold time	Low	Could be added easily
Only one long + one short per cycle	Intentional but limiting	Fine for small accounts, restrictive for large ones
No filtering of <code>bar_data_dict</code> to watched symbols	Medium	Could receive bars for symbols you care about

10. Recommended Quick Wins

1. **Implement the decision windows** (5-minute windows around :20 and :50):

```
python
minute = self._minute_of_hour(bar_data)
if minute not in (15,20,25,45,50,55):
    signal = "hold"
```
2. Add a simple time-based exit (e.g. max 4 hours) using the already-recorded `_position_entry`.
3. Make rotation configurable per direction or add more classes.
4. Expose `BAR_HISTORY_MAX` , `MAX_UNREALIZED_LOSS_PCT` , etc. as runtime parameters.

Summary Verdict

This is a **very solid, professional-grade skeleton** for a custom-signal trading bot. It is deliberately conservative (one entry per side per bar, strong risk flattening, margin safety). With the **decision-window filter added** and a good `your_signal_function` , this can be a live money maker. Without the window filter it’s basically “trade every minute on whatever the custom math says” still works, just not what the docstring promised.

Would you like me to:

- Write the missing decision-window filter code?
- Suggest a sample `your_signal_function` (e.g. RSI + volume + 20/50-min window logic)?
- Add time-based exits?
- Convert the trade journal into a backtester?

Just say the word and I’ll patch it for you.